

Table of Contents

1. Project Overview (Point-Wise)	2
Key Features Implemented:	2
Technologies Used:	2
Challenges Faced and Solutions:	3
2. Code Samples (Screenshots)	4
Create table and default admin username: admin and password:admin123.....	4
MainForm.cs – Dashboard with Role-Based Feature Access.....	4
RoomForm.cs – Manage Lecture Halls and Computer Labs.....	5
TimetableForm.cs	6
ExamController.cs	7
ER Diagram.....	8
3. How the App Works.....	Error! Bookmark not defined.
4. Features	8
5. Folder Structure Overview	9
6. Conclusion.....	10

Unicom TIC Management System - Project Submission Report

1. Project Overview (Point-Wise)

Key Features Implemented:

- Login System with role-based access for Admin, Staff, Students, and Lecturers.
- Exam and Marks Management with entry and viewing privileges by role.
- Timetable Management including Rooms, Subjects allocation.
- SQLite database with relationships between tables.
- Role-based dashboards that restrict access to appropriate features.
- Error messages and input validation included.
- Simple and intuitive WinForms user interface.

Technologies Used:

- Programming Language: C#
- Framework: WinForms (.NET Framework)
- Database: SQLite using System.Data.SQLite
- Architecture: MVC (Model–View–Controller)
- IDE: Visual Studio
- Design Patterns: MVC
- UI Elements: Buttons, ComboBoxes, DataGridViews, TextBoxes, Labels

Challenges Faced and Solutions:

- Role-Based Dashboard Visibility: Solved by checking user role after login and showing/hiding buttons accordingly.
- Room Allocation with Combo Box: Created a separate table for Rooms with room type filtering, and populated Combo Box dynamically.
- Login Validation: Added a simple user validation system querying the Users table and managing sessions based on roles.
- Database Table Creation: Implemented Migration.cs to auto-create tables if they don't exist.
- Navigation Between Forms: Managed smooth form transitions with this.Hide() and ShowDialog() methods.

2. Code Samples (Screenshots)

Create table and default admin username: admin and password:admin123

```
namespace UnicornTICManagementSystem.Repositories
{
    1 reference
    internal class Migration
    {
        1 reference
        public static void CreateTables()
        {
            string tablesQuery = @"
            CREATE TABLE IF NOT EXISTS Users(UserID INTEGER PRIMARY KEY AUTOINCREMENT,
            Username TEXT NOT NULL, Password TEXT NOT NULL, Role TEXT NOT NULL);
            CREATE TABLE IF NOT EXISTS Courses(CourseID INTEGER PRIMARY KEY AUTOINCREMENT,
            CourseName TEXT NOT NULL);
            CREATE TABLE IF NOT EXISTS Subjects(SubjectID INTEGER PRIMARY KEY AUTOINCREMENT,
            SubjectName TEXT NOT NULL, CourseID INTEGER, FOREIGN KEY (CourseID) REFERENCES Courses(CourseID));
            CREATE TABLE IF NOT EXISTS Students(StudentID INTEGER PRIMARY KEY AUTOINCREMENT, Name TEXT NOT NULL,
            CourseID INTEGER NOT NULL, FOREIGN KEY (CourseID) REFERENCES Courses(CourseID));
            CREATE TABLE IF NOT EXISTS Exams(ExamID INTEGER PRIMARY KEY AUTOINCREMENT, ExamName TEXT NOT NULL,
            SubjectID INTEGER NOT NULL, FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID));
            CREATE TABLE IF NOT EXISTS Marks(MarkID INTEGER PRIMARY KEY AUTOINCREMENT, StudentID INTEGER NOT NULL, ExamID INTEGER NOT NULL, Score INTEGER NOT NULL,
            FOREIGN KEY (StudentID) REFERENCES Students(StudentID), FOREIGN KEY (ExamID) REFERENCES Exams(ExamID));
            CREATE TABLE IF NOT EXISTS Rooms(RoomID INTEGER PRIMARY KEY AUTOINCREMENT, RoomName TEXT NOT NULL,
            RoomType TEXT NOT NULL);
            CREATE TABLE IF NOT EXISTS Timetables(TimetableID INTEGER PRIMARY KEY AUTOINCREMENT, SubjectID INTEGER NOT NULL,
            TimeSlot TEXT NOT NULL, RoomID INTEGER NOT NULL, FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID), FOREIGN KEY (RoomID) REFERENCES Rooms(RoomID));
        ";

            using (var conn = DatabaseManager.GetConnection())
            {
                SQLiteCommand cmd = new SQLiteCommand(tablesQuery, conn);
                cmd.ExecuteNonQuery();

                // Insert a default admin user if none exists
                var checkAdminCmd = new SQLiteCommand("SELECT COUNT(*) FROM Users WHERE Role = 'Admin';", conn);
                long adminCount = (long)checkAdminCmd.ExecuteScalar();
                if (adminCount == 0)
                {
                    var insertAdminCmd = new SQLiteCommand("INSERT INTO Users (Username, Password, Role) VALUES ('admin', 'admin123', 'Admin');", conn);
                    insertAdminCmd.ExecuteNonQuery();
                }
            }
        }
    }
}
```

MainForm.cs – Dashboard with Role-Based Feature Access

```
namespace UnicornTICManagementSystem.View
{
    4 references
    public partial class MainForm : Form
    {
        1 reference
        public MainForm()
        {
            InitializeComponent();
        }

        1 reference
        public void SetWelcomeText(string name, string role)
        {
            labelmainwelcome.Text = $"Welcome {name}. Your role is {role}";
            if (role == "Admin") { btnmainmarks.Visible = false; }
            else if (role == "Student")
            {
                btnmaincourses.Visible = false; btnmainstudents.Visible = false; btnmainexams.Visible = false;
                btnmaintimetable.Visible = false; btnmainroom.Visible = false; btnmainsubjects.Visible = false;
            }
            else if (role == "Lecturer") { btnmaincourses.Visible = false; btnmainstudents.Visible = false; }
            else if (role == "Staff") { }
        }

        1 reference
        private void btnmaincourses_Click(object sender, EventArgs e)
        {
            CourseForm courseForm = new CourseForm();
            courseForm.ShowDialog();
        }

        1 reference
        private void btnmainstudents_Click(object sender, EventArgs e)
        {
            StudentForm studentForm = new StudentForm();
            studentForm.ShowDialog();
        }
    }
}
```

RoomForm.cs – Manage Lecture Halls and Computer Labs

```
13 namespace UnicomTICManagementSystem.View
14 public partial class RoomForm : Form
23 private void btnroomadd_Click(object sender, EventArgs e)
27     roomController.AddRoom(room);
28     LoadRoomdata();
29 }
30
31 1 reference
32 public void EditData()
33 {
34     RoomController editController = new RoomController();
35     Room room = new Room { RoomName = textboxroomroomname.Text, RoomId = int.Parse(textroomroomID.Text), RoomType = textboxroomroomtype.Text };
36     editController.EditRoomData(room);
37 }
38 1 reference
39 public void DeleteData() {
40     RoomController deleteController = new RoomController();
41     Room room = new Room { RoomId = int.Parse(textroomroomID.Text)};
42     deleteController.DeleteRoomData(room);
43 }
44 4 references
45 public void LoadRoomdata()
46 {
47     RoomController getroomdata = new RoomController();
48     var roomData = getroomdata.GetRoomData();
49     dataroom.DataSource = roomData;
50     dataroom.Columns["RoomID"].Visible = false;
51 }
52 1 reference
53 private void btnroomedit_Click(object sender, EventArgs e)
54 {
55     EditData();
56     LoadRoomdata();
57 }
58 1 reference
59 private void dataroom_CellClick(object sender, DataGridViewCellEventArgs e)
60 {
61     if (e.RowIndex >= 0)
62     {
63         var selectedrow = dataroom.Rows[e.RowIndex].DataBoundItem as Room;
64         if (selectedrow != null)
65         {
66             textroomroomID.Text = selectedrow.RoomId.ToString();
67             textboxroomroomname.Text = selectedrow.RoomName;
68             textboxroomroomtype.Text = selectedrow.RoomType;
69         }
70     }
71 }
```

TimetableForm.cs

```
14 namespace UnicomTICManagementSystem.View
15 {
16     public partial class TimetableForm : Form
17     {
18         public void LoadTimetabledata()
19         {
20             TimetableController gettimetabledata = new TimetableController();
21             var timetableData = gettimetabledata.GettimetableData();
22             datatimetable.DataSource = timetableData;
23             datatimetable.Columns["RoomID"].Visible = false;
24             datatimetable.Columns["SubjectID"].Visible = false;
25         }
26         //=====
27         1 reference
28         public void LoadSubject()
29         {
30             var subjectList = subjectController.GetSubjectList();
31             if (subjectList.Count > 0)
32             {
33                 comboboxtimetablessubject.DataSource = subjectList;
34                 comboboxtimetablessubject.DisplayMember = "SubjectName";
35                 comboboxtimetablessubject.ValueMember = "SubjectID";
36             }
37             else
38             {
39                 labeltimetableerror.Text = "No subject found. Please add a subject first.";
40             }
41         }
42         1 reference
43         public void LoadRoom()
44         {
45             var roomList = timetableController.GetRoomList();
46             if (roomList.Count > 0)
47             {
48                 comboboxtimetableroom.DataSource = roomList;
49                 comboboxtimetableroom.DisplayMember = "RoomName";
50                 comboboxtimetableroom.ValueMember = "RoomId";
51             }
52             else { labeltimetableerror.Text = "No room found. Please add a room first."; }
53         }
54
55         0 references
56         private void comboboxtimetablessubject_SelectedIndexChanged(object sender, EventArgs e)
57         {
58             selectedSubjectID = comboboxtimetablessubject.SelectedIndex+1;
59         }
60
61         0 references
62         private void comboboxtimetableroom_SelectedIndexChanged(object sender, EventArgs e)
63         {
64             selectedRoomID = comboboxtimetableroom.SelectedIndex+1;
65         }
66     }
67 }
```

ExamController.cs

```

11 namespace UniconTICManagementSystem.Controller
12 {
13     internal class ExamController
14     {
15         2 references
16         public List<Subject> GetSubjectList()
17         {
18             List<Subject> subjectList = new List<Subject>();
19             using (var connection = DatabaseManager.GetConnection())
20             {
21                 var cmd = new SQLiteCommand("select * from Subjects", connection);
22                 var reader = cmd.ExecuteReader();
23                 while (reader.Read())
24                 {
25                     subjectList.Add(new Subject { SubjectID = reader.GetInt32(0), SubjectName = reader.GetString(1) });
26                 }
27             }
28             return subjectList;
29         }
30         1 reference
31         public void Addexam(Exam exam)
32         {
33             try
34             {
35                 using (var connection = DatabaseManager.GetConnection())
36                 {
37                     SQLiteCommand command = new SQLiteCommand("INSERT INTO Exams (ExamName,SubjectID) VALUES (@examname,@subjectID)", connection);
38                     command.Parameters.AddWithValue("@examname", exam.ExamName);
39                     command.Parameters.AddWithValue("@subjectID", exam.SubjectID);
40                     command.ExecuteNonQuery();
41                     MessageBox.Show("successfully.");
42                 }
43             }
44             catch (Exception ex) { MessageBox.Show(ex.Message); }
45         }
46         1 reference
47         public List<Student> GetStudentList()
48         {
49             List<Student> studentList = new List<Student>();
50             using (var connection = DatabaseManager.GetConnection())
51             {
52                 SQLiteCommand cmd = new SQLiteCommand("select * from Students", connection);
53                 var reader = cmd.ExecuteReader();
54                 while (reader.Read())
55                 {
56                     studentList.Add(new Student { Name = reader.GetString(1), StudentId = reader.GetInt32(0) });
57                 }
58             }
59             return studentList;
60         }
61     }
62 }

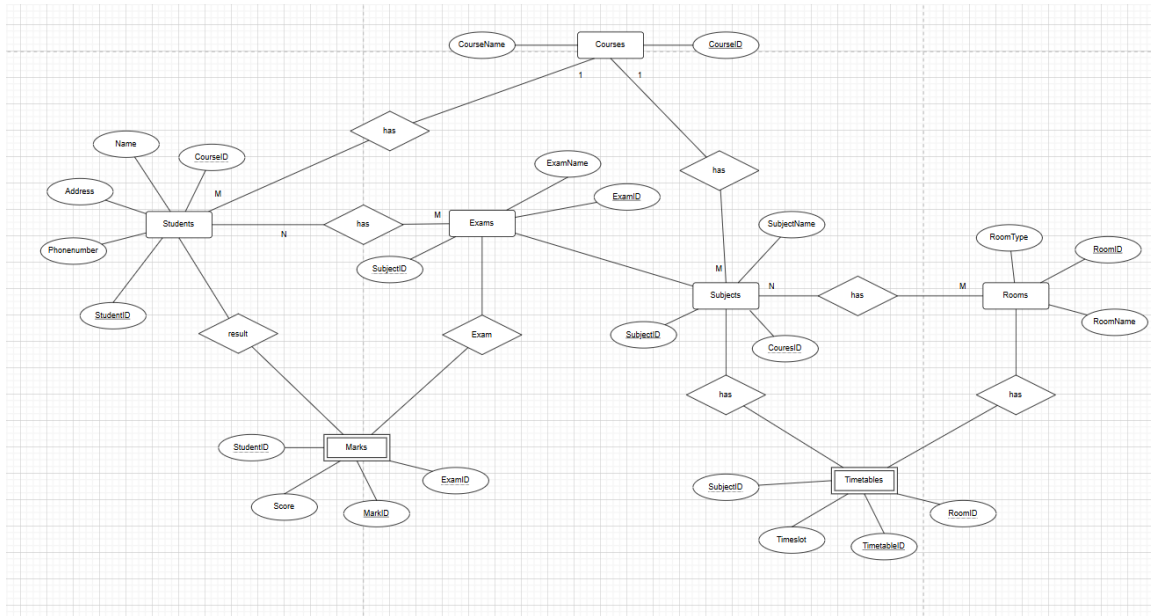
```

```

11 namespace UniconTICManagementSystem.Controller
12 {
13     internal class ExamController
14     {
15         public List<Exam> GetExamData()
16         {
17             using (var connection = DatabaseManager.GetConnection())
18             {
19                 SQLiteCommand cmd = new SQLiteCommand("select ex.ExamID, ex.ExamName, sub.SubjectID as SubjectID, sub.SubjectName as SubjectName from Exams ex Left join Subjects sub on sub.SubjectID = ex.SubjectID", connection);
20                 var reader = cmd.ExecuteReader();
21                 while (reader.Read()) { examList.Add(new Exam { ExamID = reader.GetInt32(0), ExamName = reader.GetString(1),
22                     SubjectID = reader.IsDBNull(2) ? (int?)null : reader.GetInt32(2),
23                     SubjectName = reader.IsDBNull(3) ? null : reader.GetString(3)
24                 }); }
25             }
26             return examList;
27         }
28         1 reference
29         public void EditExamData(Exam exam)
30         {
31             try
32             {
33                 using (var connection = DatabaseManager.GetConnection())
34                 {
35                     SQLiteCommand cmd = new SQLiteCommand("UPDATE Subjects SET SubjectName = @SubjectName WHERE SubjectID = @SubjectID;update Exams SET ExamName = @examname where ExamID = @examID", connection);
36                     cmd.Parameters.AddWithValue("@examID", exam.ExamID);
37                     cmd.Parameters.AddWithValue("@examname", exam.ExamName);
38                     cmd.Parameters.AddWithValue("@SubjectID", exam.SubjectID);
39                     cmd.Parameters.AddWithValue("@SubjectName", exam.SubjectName);
40                     cmd.ExecuteNonQuery();
41                     MessageBox.Show("Edit successfully");
42                 }
43             }
44             catch (Exception ex) { MessageBox.Show(ex.Message); }
45         }
46         0 references
47         public void DeleteExamData(Room room)
48         {
49             try
50             {
51                 using (var connection = DatabaseManager.GetConnection())
52                 {
53                     SQLiteCommand cmd = new SQLiteCommand("Delete from Rooms WHERE RoomID = @roomID", connection);
54                     cmd.Parameters.AddWithValue("@roomID", room.RoomID);
55                     cmd.ExecuteNonQuery();
56                     MessageBox.Show("Delete successfully");
57                 }
58             }
59             catch (Exception ex) { MessageBox.Show(ex.Message); }
60         }
61     }
62 }

```

ER Diagram



System Features and Role-Based Access

Login System: Authenticates users based on their role (Admin, Staff, Student, Lecturer).

Role Dashboards: Different dashboards are shown depending on the user role.

Data Operations: Add/Edit/Delete supported for Admin; restricted view-only for other roles.

Rooms & Timetable: Admin, Lecturer, Staff allocates labs/halls while scheduling;

Students can only view.

Exam & Marks: Staff and Lecturers can update marks; students can only view their own.

4. Features

Filter (Courses, Subjects, Students, Exams, Rooms in dropdown list)

Error Messages (e.g., “Please select a room”, “Check your username or password”)

Input Validation (e.g., Score must be 0–100)

Descriptive Mappings for Viewing Data:

- Student name | Course name
- Exam name | Subject name
- Timetable – Subject name | Room name | Time slot
- Room name | Room type
- Subject name | Course name

5. Folder Structure Overview

UnicomTICManagementSystem/

- Models
 - Course.cs
 - Subject.cs
 - Student.cs
 - Exam.cs
 - Mark.cs
 - Room.cs
 - Timetable.cs
 - User.cs
 - Lecturer.cs
- Views

- o LoginForm.cs
 - o MainForm.cs
 - o CourseForm.cs
 - o StudentForm.cs
 - o ExamForm.cs
 - o MarkForm.cs
 - o TimetableForm.cs
 - o RoomForm.cs
 - o SubjectForm.cs
 - o Staff_and_Lecturer.cs
- Controllers
 - o LoginController.cs
 - o CourseController.cs
 - o StudentController.cs
 - o ExamController.cs
 - o MarkController.cs
 - o TimetableController.cs
 - o RoomController.cs
 - o SubjectController.cs
- Repositories
 - o DatabaseManager.cs
 - o Migration.cs
- Program.cs

6. Conclusion

This project helped in learning:

- How to implement an MVC architecture in a desktop application.
- SQLite CRUD operations in C#.
- Role-based access control with a simple UI.
- Structuring a school management system from scratch.